ChatGPT

Panasonic Lumix GX880 Wi-Fi Remote Protocol – Technical Analysis

Overview and Connectivity

The Panasonic Lumix GX880 (also known as GX800/GF10 in some regions) features built-in Wi-Fi for remote control and live view. The camera can either **host its own Wi-Fi network** (direct connection) or join an existing Wi-Fi router. In **Direct Mode**, the camera acts as an access point (e.g. SSID like "GX880-xxxx"), typically with WPA2 security (password shown on camera or via WPS) [1] [2]. When connected directly, the camera usually uses a default IP (commonly `192.168.54.1` for newer Lumix) and assigns the client an IP via DHCP (e.g. `192.168.54.x`) [3] [4]. In **Wi-Fi via Router Mode**, the camera obtains an IP from the router; the smartphone app discovers the camera using SSDP/UPnP broadcasts. In both cases, **no additional login** is required beyond the Wi-Fi credentials – instead, Panasonic employs an *application-layer pairing handshake* to authorize new remote devices.

Once the camera's Wi-Fi is enabled in *"Remote Shooting"* mode, a control session can be initiated from a phone or computer. Under the hood, the GX880 (like other Lumix models) exposes an HTTP-based control interface. The controlling device sends **HTTP GET requests** to the camera's IP (port 80) at the endpoint `/cam.cgi` with various query parameters for different commands [5]. This HTTP API is not officially documented by Panasonic, but has been extensively reverse-engineered by the community. Multiple open-source projects and tools (in Python, Java, C/C++, etc.) leverage this protocol [6], and recent versions of libgphoto2 even introduced a Wi-Fi Lumix driver using these HTTP calls (with many functions supported, though as of that writing continuous capture was still being worked on) [7].

Initial Handshake and Pairing (Authorization Mechanism)

Before sending any control commands, a one-time pairing handshake is required for new devices. The Lumix GX880 implements a simple access control step to prevent unauthorized control. The first time a new device attempts to connect, the camera will prompt on its screen to authorize the remote device. This is triggered by a special HTTP request:

- **Request Access (`accctrl`):** The controller sends `http://<camera_ip>/cam.cgi?mode=accctrl&type=req_acc&value=<ID>&value2=<Name>` [8]. Here `<Name>` is a string identifying the device (e.g. a phone model or app name), and `<ID>` is an identifier often formatted as a GUID. If the device has no prior relationship, a dummy or "0" value can be used for `<ID>` to initiate pairing [9] [10]. For example, a PC might send:

```
GET /cam.cgi?mode=accctrl&type=req_acc&value=0&value2=MyComputer
```

On first use, the camera responds with a status that causes it to display a confirmation on the camera LCD (showing the device name). The user must **accept/authorize** the connection on the camera. Upon

acceptance, the camera returns an `ok` along with a unique identifier token. For instance, in one reverse-engineering example the camera's reply included a GUID like "4D454930-0100-1000-8001-024500021C98" which the app then uses for subsequent sessions [11]. The camera also internally **stores the device** (e.g. listing the device name in its Wi-Fi history) as an authorized controller [12].

- **Timeout Behavior:** Notably, after authorizing a new device, the camera might show a *"connection failed"* or timeout message on the first attempt [13]. This is expected – the initial `req_acc` handshake doesn't fully open the session by itself. The device is now remembered/whitelisted, so future connections can proceed without another on-camera prompt [12].

For **subsequent connections**, the controller should send the same `accctrl` request again (this time the camera will immediately return OK since the device ID is recognized), and then continue with normal operation [14] [15]. In practice, the official Panasonic Image App performs this handshake behind the scenes: it sends the camera a **device name** and **unique ID**, waits for the user to confirm on first use, and then uses the saved credentials thereafter [16] [17]. If the device sends an unrecognized name/ID later (e.g. a third-party app with a different signature), the camera will respond with an error like "err-unsuitable-app" and refuse control until authorized [9]. (Advanced users have found that spoofing the official app's identifier can bypass prompts in some cases [18], but ordinarily one should simply authorize the device properly.)

Example: In one community example with a GH5 (which uses the same protocol generation as GX80/GX880), a PC user performed:

1. `GET /cam.cgi?mode=accctrl&type=req_acc&value=4D454930-0100-1000-8001-020A000270B8&value2=D2304` – Camera asked for authorization for "D2304" on its screen, user accepted. Camera then timed out (one-time).
2. On subsequent connect: Repeat the same `req_acc` request (no prompt this time, camera returns OK), then send `GET /cam.cgi?mode=camcmd&value=recmode` to put the camera into record control mode [19]. At this point the camera's LCD shows "Under remote control" and live view can be streamed. This GH5 then accepted further commands (e.g. changing shutter speed via `setsetting`) without issue [20].

Thus, the **pairing flow** for a new GX880 controller is: connect to camera Wi-Fi → send `accctrl` request → user confirms on camera → camera returns "ok" (with ID) → send a command to enter remote mode. After that, the session is established. The camera's Wi-Fi menu will list the device name as a known connection, enabling seamless future use [12].

Entering Remote Control Mode

After authorization, the camera may start in a **playback state** by default. The controller should explicitly switch the camera to **recording (capture) mode** for live view and capture control. This is done by an HTTP command:

- `cam.cgi?mode=camcmd&value=recmode` – Puts the camera into record/shoot mode for remote control [21]. The camera's display typically turns on or shows a "Remote Control" indicator after this.

Conversely, sending `value=playmode` will switch the camera into playback/browse mode remotely [22] [23] (useful for image download functions).

In practice, if you connect while the camera is already in the *Remote Shooting* Wi-Fi function, it might automatically be ready in recmode. But it's good practice to issue the `recmode` command to ensure the camera is in the correct state to accept capture commands. (On some models, starting a live view stream implicitly puts the camera in rec mode as well.)

Once in "record" mode, the GX880 will stay active as long as commands or live view keep coming. Note that the camera might still enforce its power-save timer – if the screen goes black due to inactivity, you may need to half-press the physical shutter or send a dummy command (like a focus command) to wake it [24] [25]. When finished, the connection can be terminated by sending `camcmd=terminate` (or simply disconnecting, which the camera will eventually detect).

Command Interface and Control Structures

The Lumix Wi-Fi API is built around simple **query strings**. The general format is:

```
http://<camera_ip>/cam.cgi?mode=<action>&<param>=<value>[&<param2>=<value2>...]
```

Key modes and their usage include:

- **Camera Commands (`mode=camcmd`):** For high-level actions – taking photos, starting/stopping video, toggling camera modes, etc. For example:
- `...cam.cgi?mode=camcmd&value=capture` – Triggers a still photo capture (equivalent to pressing the shutter) [26] [5].
- `...cam.cgi?mode=camcmd&value=video_recstart` – Begin video recording [27]; similarly `video_recstop` to stop [28].
- `...cam.cgi?mode=camcmd&value=zoomup` / `zoomdown` – Zoom in/out (for power zoom lenses) *(exact values may vary; some use camctrl for zoom).*
- `...cam.cgi?mode=camcmd&value=focuspress` – Half-press focus (on supported models).
- `...cam.cgi?mode=camcmd&value=oneshot_af` – Perform a one-shot autofocus operation [29].
- `...cam.cgi?mode=camcmd&value=burst_start` / `burst_stop` – Continuous shooting control (if available).

- `...cam.cgi?mode=camcmd&value=flip` – Flip selfie screen mode, etc. (Many minor commands exist – a comprehensive list was gleaned from the camera's menu XML, as described below.)

- **Set/Get Settings (`mode=setsetting` or `getsetting`):** For adjusting specific camera parameters and retrieving settings. These use a `type=<setting_name>` and `value=<setting_value>` format.

- **Setting Exposure Parameters:** e.g. `...mode=setsetting&type=iso&value=400` sets ISO to 400, or `value=auto` for Auto ISO [30]. Shutter speed and aperture are set via coded values (the camera uses internal units). For instance, `type=shtrspeed&value=2390/256` set the GH5's shutter to 1/640s [31]. Aperture is set with `type=focal` (for historical reasons) – e.g.

`value=392/256` corresponded to F1.7 on a certain lens, while `2304/256` was F22 [32] [33]. These values are 1/256 increments of some internal scale (the mapping can be obtained via the camera's menu info; see below).

- **Other Settings:** Practically any option in the camera's menus can be changed: `focusmode` (e.g. afs/aff/afc/mf) [34], metering mode (`lightmetering=multi/center/spot`) [35], white balance, drive mode, image format, video recording mode (`videoquality=mp4_25p_100mbps_4k`, etc.) [36], picture style/filters (`colormode=vivid`, `filter_setting=retro`, etc.) [37] [38], exposure compensation (`exposure=+5/3` for +1⅔ EV, etc.) [39], image resolution, flash mode, and so on. The camera exposes these settings by logical names, making automation straightforward.

- **Get Setting:** You can query current values with `mode=getsetting&type=<setting>` for many parameters (returns an XML or comma-separated value). For example, `getsetting&type=iso` might return the current ISO value or mode.

- **Camera Control (`mode=camctrl`):** These are used for continuous or interactive controls, often with two parameters. For example:

- **Touch Focus/Tracking:** The GX880 allows setting a focus point via touch coordinates. The app can simulate this by `camctrl&type=touch_trace`. For instance, `...mode=camctrl&type=touch_trace&value=start&value2=264/431` begins a touch focus at screen coordinate (264,431), followed by `continue` events as you "drag" and a final `stop` with coordinates [40]. This lets a remote app tell the camera where to focus or to change the focus point dynamically.
- **Manual Focus Drive:** Electronic focus (for lenses with focus-by-wire) can be controlled with `camctrl&type=focus&value=wide-normal` / `wide-fast` / `tele-normal` / `tele-fast` [41] [29]. "Wide" vs "Tele" here refer to adjusting focus toward infinity or close focus (borrowing the terminology of turning the focus ring towards the wide vs tele end on power zoom lenses), and "normal" vs "fast" pick the speed/step size. Essentially, these commands nudge the focus motor in small or large increments – useful for remote manual focus adjustments.
- **Triggering Camera UI Elements:** e.g. `camctrl&type=asst_disp&value=current_auto&value2=mf_asst/0/0` can toggle focus peaking or magnified assist [42], or `camctrl&type=menu_entry` to simulate a menu button press [43]. These are more specialized and often reverse-engineered by monitoring the official app.

- **Zoom (alternative):** Some Lumix use `camctrl` for zooming via `type=zoom` or similar, though in many models zoom is a `camcmd`. (GX880's kit lens is manual zoom, so not applicable.)

- **Information Queries (`mode=getinfo` and `mode=getstate`):** These endpoints return XML data with various camera status and capabilities:

- `...mode=getinfo&type=allmenu` – Dump of *all menu items* and options, in multiple languages [44] [45]. This includes valid values for settings and many command names. The GH3 reverse engineering team found this extremely useful, as it essentially exposes the camera's internal menu structure and supported commands.
- `...mode=getinfo&type=curmenu` – XML of the *current state* of settings (which menu items are enabled/disabled given the current mode, etc.) [46] [47]. For example, it can tell you if a certain

feature is available in the current mode (e.g. if not in P mode, “program_shift” might show enabled="no" in the XML) [47] .

- `...mode=getinfo&type=lens` – Provides a string of lens info: focal length range, current focal length, aperture range, stabilization, etc. [48] . For instance, a 14-42mm power-zoom might return a comma-separated string with focal lengths and other flags [48] .
- `...mode=getinfo&type=capability` – (On some models) returns an XML with supported features/capabilities of the camera (e.g. does it support focus stacking, 4K photo, etc.).
- `...mode=getstate` – Returns a brief status (e.g. whether the camera is currently capturing, in what mode, battery level, etc.). This is often a simple comma-separated list or XML string polled periodically by the app to know if the camera is busy.
- **Heartbeat:** The Image App also sends periodic no-op requests (like `getstate` or small commands) to keep the connection alive. If no commands are received for a while, the camera may drop the Wi-Fi link to save power.

All responses are typically plain text or XML. For example, a successful command returns a snippet like `<camrply><result>ok</result></camrply>` (or `ok` followed by some data) [49] . An error will return `<camrply><result>err</result><detail>...</detail></camrply>` with an error code or message (e.g. “err,busy” if the camera is doing something that prevents the command).

Practical insight: You can test basic commands by simply entering the URLs in a web browser (when connected to the camera’s Wi-Fi). For instance, browsing to `http://192.168.54.1/cam.cgi?mode=camcmd&value=capture` will immediately fire the shutter on a connected GX880 [50] [51] . The browser will show an “ok” or an XML result in return. Many developers have automated this using scripts or libraries (e.g. using `curl` or Python requests). The Python module **python_lumix_control** wraps these HTTP calls into easy functions, as does a community Java app **Lumix Link Desktop**, among others [6] . These tools internally implement the same sequence: perform the `accctrl` handshake if needed, then send camcmd/setsetting calls to control the camera.

Live View Video Streaming

One of the most useful features is the ability to get a **live view feed** from the camera’s sensor. The GX880’s live view is streamed as an MJPEG video over UDP. The steps to enable it:

- **Start Stream:** Send an HTTP GET request to `http://<camera_ip>/cam.cgi?mode=startstream&value=<port>` [52] . For example: `...startstream&value=49152` to request the stream on UDP port 49152. The camera will then initiate a UDP connection, sending a continuous stream of packets to the client’s IP on that port [53] . (The camera determines the destination IP implicitly from the TCP/IP connection of the HTTP request – i.e. it streams to whichever host made the request on the given port.)

- **Stream Format:** The stream is **not RTP or RTSP**, but rather a custom raw UDP sequence. It was discovered to be Motion JPEG: essentially a series of JPEG images one after another, preceded by small header blocks [53] [54] . Each frame’s JPEG data starts with the standard JPEG SOI marker `0xFF 0xD8` and ends with `0xFF 0xD9` (EOI marker) [55] . Between frames, the camera inserts a fixed-size or variable-size header. Early research on the GH3 found a 144-byte repeating header between JPEG frames [56] , though later models showed the header size can vary (it can change with camera state – e.g. AF active, mode dial changes – the Java code provided by the GH3 hacker

adjusted for this) [57]. The header contains no standard RTP or JPEG payload header; it seems to be proprietary, possibly carrying sequence numbers or timestamps. However, since each frame is delineated by the `FF D8 ... FF D9` JPEG markers, one can parse out the images easily.

- **Frame Rate & Resolution:** The stream provides a live view in (approximately) VGA resolution with a modest frame rate. On older models (GH3/GH4), the resolution was around 640x480 or 320x240 depending on conditions [58]. Newer models sometimes offer slightly higher resolution live view (some report around 640x480 or 640x360 in 16:9). The frame rate is typically around 30 fps or lower. It is *not* a high-bandwidth video stream – it's meant for preview, so expect some latency and lower quality. The bit rate is limited by the camera's processing; the stream will pause if the camera is busy (e.g. when capturing a photo, the live view may freeze briefly).

- **Consuming the Stream:** To use the live view, the client should listen on the specified UDP port for incoming data. One can write a simple UDP socket listener that assembles packets into a byte stream. Because it's MJPEG, feeding the byte stream into a player or decoder that understands Motion JPEG is possible. For example, some have written programs to strip out the custom headers and pipe the JPEG frames to a display. The GH3 community created a Java UDPServer that received the stream and displayed the images [59] [60]. You can also save the JPEG frames to disk or process them for computer vision tasks. Keep in mind that the UDP packets may fragment a JPEG frame; you need to reassemble them in order. There is no retransmission (UDP), but since it's local network it's usually reliable enough. If needed, the camera can be asked to resend by restarting the stream.

- **Stopping the Stream:** There isn't a specific "stopstream" command in the HTTP API (beyond perhaps a generic terminate). Typically, the stream stops when the remote controller disconnects or when the camera is commanded to switch mode. In practice, to stop streaming you can send `cam.cgi?mode=camcmd&value=playmode` (to exit record mode) [61] or simply close the app – the camera should detect the control session has ended after a timeout and stop the UDP feed. On the GH series, issuing `playmode` was even used as a hack to stop video recording remotely [62] (since there wasn't a dedicated video stop command on GH3, but GX880 has `video_recstop` which is cleaner).

Stream Example: A user who captured the GH3 stream noted the UDP data contained JPEG images separated by a consistent header, and the last two bytes of each frame were `FF D9` confirming JPEG format [53]. By extracting those and ignoring the 144-byte chunks, they successfully viewed the live view on a PC as a video [63]. This was effectively an MJPEG over UDP stream. No standard RTP headers (e.g., no 0x80 0x60... RTP sequences) were present, so standard media players wouldn't directly open it without a custom parser.

Wi-Fi Network Configuration and UPnP Services

Beyond the HTTP `cam.cgi` interface for live control, the Lumix GX880 also runs standard **UPnP (Universal Plug and Play) / DLNA services** for media transfer. In *Playback/Playmode*, the camera acts as a **DLNA server**, allowing browsing and transfer of images wirelessly. The Panasonic Image App leverages this to show the camera's gallery and download photos. Technically, the camera advertises a UPnP MediaServer on a high-numbered port (e.g. port 60606). For example, GX80/GX880 devices host a device descriptor at `http://<camera_ip>:60606/<uuid>/Server0/ddd` [64] – this is an XML describing the "Lumix Digital

Camera" media server. It includes Content Directory service endpoints (for listing files) and ConnectionManager, etc. The smartphone (as a DLNA control point) can send **SOAP commands** to browse folders and fetch files. Community developers captured these SOAP messages: for instance, an `X_GetList` action (Panasonic's custom browsing command) is used to retrieve the list of images, and an `X_GetContent` to transfer a file [65]. The GX880's UPnP implementation closely follows standard Digital Media Server profiles [66].

Usage: If you send the camera to *playmode* (`camcmd&value=playmode`), you can then navigate the file system. The content can be accessed via HTTP: the camera's images are usually available under a URL like `http://<camera_ip>:60606/...(some path)...`. The tricky part is obtaining the correct URLs, which is where the SOAP queries come in. The Image App automates this, but it is possible to replicate with any DLNA browser. In fact, you can point a standard DLNA client at the camera and it should recognize a device named "Lumix <model>" sharing photos. This is beyond remote **control** per se, but it's part of the Wi-Fi protocol suite the camera supports.

Auto-Discovery: On the same note, when the camera is connected to a normal Wi-Fi network, it likely uses **SSDP (Simple Service Discovery Protocol)** to announce itself. The Panasonic apps can discover the camera's IP by listening for SSDP broadcasts of the Lumix's UPnP UUID and then querying the `.../ddd` descriptor [67] [68]. In direct mode, discovery is trivial since the camera is at a known IP. But in a home network scenario, this allows the app to find the camera without manual IP entry. The Python lumix control module in fact uses an SSDP library to auto-discover Lumix cameras [69]. Alternatively, some developers just scan for open port 80 on the local subnet after the camera joins – since the camera often uses a predictable name or has the MAC OUI of Panasonic.

Security and Authentication

Aside from the initial pairing handshake, there is **no further authentication layer** (no logins or challenge/response). Once a device is authorized and possesses the camera's assigned ID, it can initiate control sessions by sending the correct `accctrl` request. It appears the camera will accept commands from any host on the Wi-Fi network that presents a known device ID. This means the security mainly relies on the Wi-Fi network itself (and the user's consent during pairing). In direct mode, the WPA2-PSK password (or physical access to press "OK" on the camera) is the gatekeeper. In router mode, as long as your network is secure, that suffices – but theoretically if an attacker is on the same network and knows the protocol (and if the camera is in remote mode awaiting a controller), they could attempt to send commands. The risk is low and mitigated by the necessity of pairing (an attacker wouldn't have your camera's GUID unless they sniffed it or had prior access).

One interesting tidbit: the official app's *device ID* often begins with `4D454930-...` which is "MEI0" in ASCII (possibly meaning "Media Engine" or just a Panasonic code) [70]. This suggests the GUID might be derived from the camera's identity (the camera could be sending "MEI0..." as part of its self-identification, which the app then uses). In logs, we also see value2 often set to phone model names (e.g. `SM-G903` for a Samsung phone, or `DMC-CM1` which was Panasonic's own camera-phone) [70] [71]. The camera likely records that string as the "device name." For example, after pairing, the GH5 showed "D2304" in its known connections list [72] – exactly the value2 used. Therefore, **value2 (device_name)** is purely a label, and **value (GUID)** is the actual credential.

To re-pair or revoke devices, one can use the camera's menu (under Wi-Fi settings > "History" or "Authorized devices") to delete entries. Then that device would need to do the handshake again.

Practical Implementation Resources

Developers aiming to integrate GX880 control can reference several community projects. The GitHub repositories **Lumix Link Desktop (Java)**, **python_lumix_control (Python)**, **qtpanaremote (C++/Qt)**, and an ASCOM driver (VB.NET) all implement this protocol [6]. They provide working code for the command sequences, including the live view decoding. For instance, the Lumix Link Desktop README describes the security handshake and suggests sending a `req_acc` URL in a browser to pre-authorize if you get an "unsuitable app" error [9]. The **lumixproto** project by @cleverfox specifically focused on the GX80 and lists many commands and their effects (from focus steps to changing filters) [73] [74], which aligns with GX880 behavior.

In usage, controlling the camera can be as simple as using `curl` commands. A Raspberry Pi forum user demonstrated capturing with an LX15 (similar protocol) by issuing `curl "http://192.168.54.1/cam.cgi?mode=camcmd&value=capture"` from a script [75] [76]. Likewise, OctoPrint/OctoLapse users have scripted Lumix Wi-Fi control to automate photography: their scripts set the device name, do the accctrl handshake, then call `capture` for each photo in a timelapse [77] [78]. These real-world examples confirm that with the proper sequence, even a headless device can remote-trigger a Lumix over Wi-Fi reliably.

Conclusion

In summary, the Panasonic Lumix GX880's Wi-Fi protocol uses a simple HTTP-based API for remote operations and a UDP MJPEG stream for live view. The workflow is: join Wi-Fi → perform one-time authorization (`accctrl`) → switch to recmode → send commands (capture, focus, settings) as needed → optionally start liveview stream for video feed. An XML-based UPnP/DLNA service complements this for media browsing when in playback mode. The protocol offers comprehensive control: virtually all settings (exposure, focus, zoom, drive modes, etc.) and actions can be manipulated remotely [79] [80], which is invaluable for automation, scripting, or integrating the camera into remote setups. While Panasonic doesn't officially document it for end-users, the system has been **well charted by enthusiasts**. Armed with these findings – and using existing libraries or the references cited – developers can integrate GX880 remote control and live view streaming into custom applications with relative ease. The GX880 essentially becomes a controllable IP camera with DSLR capabilities, once you speak its "language" of `cam.cgi` commands and JPEG streams.

Sources: The above analysis is drawn from reverse-engineering reports and community documentation of Panasonic's Wi-Fi protocol – notably the Personal View forum deep-dive on the GH3 [5] [53], code and notes from the Lumix Link Desktop project [9] [19], the cleverfox/lumixproto repository for GX80 [81] [82], and various user implementations and discussions [13] [77]. These sources detail the HTTP command structures, the MJPEG stream format, and the initial pairing/authentication mechanism, all of which apply to the GX880's wireless interface. The consistency of results across models (GH3/GH5/GX80/LX15/etc.) confirms that Panasonic uses a unified protocol across the Lumix line [83] [84], making these insights broadly applicable to the GX880.

---

1 2 Connecting The Camera And Another Device Directly (Direct Connection) - Panasonic DX-GX800 Operating Instruction And Advanced Features [Page 272] | ManualsLib
https://www.manualslib.com/manual/1233288/Panasonic-Dx-Gx800.html?page=272

3 4 21 49 50 51 75 76 Shooting Panasonic Lumix camera through wifi and Python scripts - Raspberry Pi Forums
https://forums.raspberrypi.com/viewtopic.php?t=262455

5 24 25 26 44 45 46 47 48 52 53 54 55 56 57 59 60 61 62 63 65 66 67 68 Control your GH3 from a Web Browser - Now with video ! - Personal View Talks
http://www.personal-view.com/talks/discussion/6703/control-your-gh3-from-a-web-browser-now-with-video-/p1

6 58 83 Lumix HTTP/Wifi protocol · Issue #409 · gphoto/libgphoto2 · GitHub
https://github.com/gphoto/libgphoto2/issues/409

7 News - gPhoto
http://www.gphoto.org/news/

8 22 23 27 28 29 30 32 33 34 35 36 37 38 39 40 41 42 43 64 73 74 79 80 81 82 GitHub - cleverfox/lumixproto: Lumix GX80 wifi protocol reverse
https://github.com/cleverfox/lumixproto

9 18 84 GitHub - peci1/lumix-link-desktop: A unofficial desktop version of the Panasonic Lumix Link app for remote control of your Lumix camera.
https://github.com/peci1/lumix-link-desktop

10 libgphoto2/camlibs/lumix/lumix.c at master · gphoto/libgphoto2 · GitHub
https://github.com/gphoto/libgphoto2/blob/master/camlibs/lumix/lumix.c

11 70 77 78 Octolapse connect to camera but give Error - Webcams - OctoPrint Community Forum
https://community.octoprint.org/t/octolapse-connect-to-camera-but-give-error/24605

12 13 14 15 16 17 19 20 31 72 A quick favour from someone with a GH5 if you can help? - Cameras - EOSHD Forum
https://www.eoshd.com/comments/topic/24986-a-quick-favour-from-someone-with-a-gh5-if-you-can-help/

69 GitHub - palmdalian/python_lumix_control: Python module for controlling Panasonic wifi enabled cameras
https://github.com/palmdalian/python_lumix_control

71 remote wireless control GX7 by google chrome | DPReview Forums
https://www.dpreview.com/forums/threads/remote-wireless-control-gx7-by-google-chrome.3875367/